

UNIT-IV Exception Handling and I/O Programming

Exception Handling-

An exception is a problem that arises during the execution of a program .

A C# exception is a response to an exceptional circumstance that arises while a program is running, such as an attempt to divide by zero.

Exception handling is used to make sure that the program flow is continued even if there is a runtime error. Exceptions are the objects that are thrown while encountering any runtime error.

If the exceptions are not handled, then C# will produce an exception message and will terminate the program abnormally, this is where exception handling comes into the picture.

The following example shows program execution with an exception. As you can see, in the below code, we are dividing an integer number by 0 which is not possible in mathematics. So, it will throw the Divide By Zero Exception in this case. The statements which are present before the exception-causing statement i.e. before `c = a / b;` is executed and the statements which are present after the exception-causing statement will not be executed.

```
using System;
namespace ExceptionHandlingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int a = 20;
            int b = 0;
            int c;
            Console.WriteLine("A VALUE = " + a);
            Console.WriteLine("B VALUE = " + b);
            c = a / b;
            Console.WriteLine("C VALUE = " + c);
            Console.ReadKey();
        }
    }
}
```

Output:

```
A VALUE = 20  
B VALUE = 0
```

After printing the above value it will give us the below error

```
class Program  
{  
    static void Main(string[] args)  
    {  
        int a = 20;  
        int b = 0;  
        int c;  
        Console.WriteLine  
        Console.WriteLine  
        c = a / b;  
        Console.WriteLine  
        Console.ReadKey()  
    }  
}
```

Exception Unhandled

System.DivideByZeroException: 'Attempted to divide by zero.'

[View Details](#) | [Copy Details](#)

▶ Exception Settings

Keywords for Exception Handling

Exceptions provide a way to transfer control from one part of a program to another. C# exception handling is built upon four keywords: **try**, **catch**, **finally**, and **throw**.

- **try** – A try block identifies a block of code for which particular exceptions is activated. It is followed by one or more catch blocks.
- **catch** – A program catches an exception with an exception handler at the place in a program where you want to handle the problem. The catch keyword indicates the catching of an exception.
- **finally** – The finally block is used to execute a given set of statements, whether an exception is thrown or not thrown. For example, if you open a file, it must be closed whether an exception is raised or not.
- **throw** – A program throws an exception when a problem shows up. This is done using a throw keyword.

Syntax-

```

try
{
    // put the code here that may raise exceptions
}
catch
{
    // handle exception here
}
finally
{
    // final cleanup code
}

```

Exception Classes In C#

All the exceptions in the C# are derived directly or indirectly from the **System.Exception** class.

Let us have a brief look at some of the most common exceptions:

Table 18.1 Common C# exceptions

EXCEPTION CLASS	CAUSE OF EXCEPTION
SystemException	A failed run-time check; used as a base class for other exceptions
AccessException	Failure to access a type member, such as a method or field
ArgumentException	An argument to a method was invalid
ArgumentNullException	A null argument was passed to a method that does not accept it
ArgumentOutOfRangeException	Argument value is out of range
ArithmeticException	Arithmetic over-or underflow has occurred
ArrayTypeMismatchException	Attempt to store the wrong type of object in an array
BadImageFormatException	Image is in the wrong format
CoreException	Base class for exceptions thrown by the runtime
DivideByZeroException	An attempt was made to divide by zero
FormatException	The format of an argument is wrong
IndexOutOfRangeException	An array index is out of bounds
InvalidCastException	An attempt was made to cast to an invalid class
InvalidOperationException	A method was called at an invalid time
MissingMemberException	An invalid version of a DLL was accessed
NotFiniteNumberException	A number is not valid
NotSupportedException	Indicates that a method is not implemented by a class
NullReferenceException	Attempt to use an unassigned reference
OutOfMemoryException	Not enough memory to continue execution
StackOverflowException	A stack has overflowed

Exception Handling in C# using Try-Catch Implementation

```
namespace ExceptionDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Enter Values For a and b....");
            int a = Convert.ToInt32(Console.ReadLine());
            int b = Convert.ToInt32(Console.ReadLine());
            try
            {
                int c = a / b;
                Console.WriteLine("Result=" + c);
            }
            catch(Exception e)
            {
                Console.WriteLine(e.Message);
            }

            Console.WriteLine("EOF PROGRAM.....");
            Console.ReadKey();
        }
    }
}
```

Output-

```
Enter Values For a and b....
10
0
Attempted to divide by zero.
EOF PROGRAM.....
```

Throwing Custom Exceptions-

An exception that is raised explicitly under a program based on our own condition (i.e. user-defined condition) is known as an application exception or Custom Exception.

As a programmer, we can raise application exceptions at any given point in time.

To raise an Application Exception in C#, we need to adopt the following process. First, we need to create a custom Exception class by inheriting it from the Parent Exception class and then we need to create an instance of the Custom Exception class and then we need to throw that instance.

```
namespace CustomExceptionDemo
{
    class MyException:ApplicationException
    {
        public void showerror()
        {
            Console.WriteLine("EXCEPTION RAISED....ODD
NUMBER IS NOT ALLOWED....");
        }
    }
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Enter only even number....");
            int n = Convert.ToInt32(Console.ReadLine());
            try
            {
                if (n % 2 != 0)
                    throw new MyException();
                else
                    Console.WriteLine("Square of Even
number..." + n + "IS " + (n * n));
            }
            catch(MyException e)
            {
                e.showerror();
            }

            Console.ReadKey();
        }
    }
}
```

Output-

Enter only even number....

7

EXCEPTION RAISED....ODD NUMBER IS NOT ALLOWED....

Multiple Catch Blocks-

In C#, you can use more than one catch block with the try block. ***Generally, multiple catch blocks is used to handle different types of exceptions means each catch block is used to handle different type of exception.***

If you use multiple catch blocks for the same type of exception, then it will give you a compile-time error because C# does not allow you to use multiple catch block for the same type of exception. A catch block is always preceded by the try block.

In general, *the catch block is checked within the order in which they have occurred in the program. If the given type of exception is matched with the first catch block, then first catch block executes and the remaining of the catch blocks are ignored. And if the starting catch block is not suitable for the exception type, then compiler search for the next catch block.*

Syntax:

```
try {
```

```
// Your code
```

```
}
```

```
// 1st catch block
```

```
catch(Exception_Name)
```

```
{
```

```
// Code
```

```
}
```

```
// 2nd catch block
```

```
catch(Exception_Name)
```

```
{
```

```
// Code  
  
}
```

EX.

```
namespace ConsoleApplication74  
{  
    class MyOwnException:ApplicationException  
    {  
        public void showerror()  
        {  
            Console.WriteLine("Enter only EVEN Number....");  
        }  
    }  
  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            Console.WriteLine("Enter Values For a and b");  
            int a = int.Parse(Console.ReadLine());  
            int b = int.Parse(Console.ReadLine());  
  
            try  
            {  
                if (b % 2 != 0) throw new MyOwnException();  
                else Console.WriteLine("DIVISION=" + (a / b));  
                if (b == 0) throw new DivideByZeroException();  
                else Console.WriteLine("DIVISION=" + (a / b));  
            }  
  
            catch (MyOwnException Me)  
            {  
                Me.showerror();  
            }  
            catch (DivideByZeroException de)
```

```
        {  
            Console.WriteLine(de.Message);  
        }  
        catch (Exception e)  
        {  
            Console.WriteLine(e.Message);  
        }  
        Console.WriteLine("END OF PROGRAM");  
        Console.ReadKey();  
    }  
}
```

Output-1-

Enter Values For a and b

10

3

Enter only EVEN Number....

END OF PROGRAM

Output-2-

Enter Values For a and b

10

0

Attempted to divide by zero.

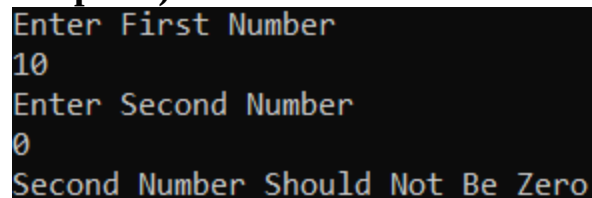
END OF PROGRAM

Ex-2)

```
using System;  
namespace ExceptionHandlingDemo  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            int Number1, Number2, Result;  
            try
```



```
{  
Console.WriteLine("Enter First Number");  
Number1 = int.Parse(Console.ReadLine());  
  
Console.WriteLine("Enter Second Number");  
Number2 = int.Parse(Console.ReadLine());  
  
Result = Number1 / Number2;  
  
Console.WriteLine("Result: "+Result);  
}  
catch (DivideByZeroException DBZE)  
{  
Console.WriteLine("Second Number Should Not Be Zero");  
}  
catch (FormatException FE)  
{  
Console.WriteLine("Enter Only Integer Numbers");  
}  
catch (Exception ex)  
{  
Console.WriteLine("Generic Catch Block...");  
}  
Console.ReadKey();  
}  
}  
}
```

Output-1)

```
Enter First Number  
10  
Enter Second Number  
0  
Second Number Should Not Be Zero
```

Output-2)

```
Enter First Number
10
Enter Second Number
abc
Enter Only Integer Numbers
```

Output-3)

```
Enter First Number
23456789234
Generic Catch Block...
```